QDRANT · REDIS · FEAST

Полный технологический стек через адаптеры

Порты, локальные реализации, настоящие сервисы и доказательства для жюри.

20 октября 2026
Согласованная спецификация

Команда 3-5 человек
Окно 48-72 часа

Приложение не реализовано. Статические проверки документации и контрактов отделены от будущих испытаний runtime.

Редакция HACKATHON-1.5. Qdrant, Redis, Feast, MLflow, Airflow, Evidently, vLLM, Kafka, S3, Prometheus, Grafana и Kubernetes сохранены как именованные архитектурные компоненты с назначением, владельцем и способом подключения. Java/Spring, React/TypeScript, PostgreSQL и Python ML остаются основой.

Это спецификация будущей реализации. Порты, адаптеры и Docker-профили ниже спроектированы; исполняемых адаптеров, контейнеров приложения и результатов их испытаний в этой поставке нет. Все записи каталога имеют статус SPECIFIED_NOT_IMPLEMENTED.

1. Как это устроено

Бизнес-модуль вызывает небольшой интерфейс способности: поиск, получение признаков, работа с сессией или чтение файла. В точке сборки Java/Python-приложения выбирается ровно одна реализация интерфейса. Для обычного ПК выбирается локальный адаптер; для интеграционного стенда - адаптер настоящего сервиса. SDK не импортируется из доменной логики. Общие JSON DTO описывают семантику на межпроцессных границах; интерфейс Python не превращается в Java-библиотеку.

Такой подход сохраняет технологии в структуре будущего проекта: backend/ports и backend/adapters/{local,redis,kafka,s3}; ml/ports и ml/adapters/{local,qdrant,feast,mlflow,airflow,evidently,vllm}; deploy/profiles; tests/adapter-contracts. Каталоги настоящих SDK-адаптеров на старте могут содержать интерфейс, README, план подключения и явно отключённый каркас. Каркас выбрасывает ProviderNotImplemented при попытке выбора; он не возвращает успешный фиктивный результат.

Локальный адаптер выполняет полезную работу: ищет embeddings, читает реальные снимки признаков, сохраняет сессии или считает DQ. Контрактная заглушка отдаёт подготовленный ответ для независимой разработки и тестов. Эти два режима различаются в конфигурации и отчёте защиты. Четыре согласованные ML-функции в итоговом MVP требуют настоящих моделей и вычислений.

2. Профили запуска

Профиль	Привязка способностей	Условие готовности
local-mvp	6 контейнеров; локальные адаптеры всех компонентов; реальные risk, рекомендации, OCR, RAG	Обязательный CPU-профиль; общий путь, данные и безопасность проверены
local-integrations	Пример: настоящий Redis для кеша, Qdrant для индекса, MLflow для registry; прочие привязки локальные	Дополнительная задача P27; проверены SDK, серверы, ресурсы и общий контракт. Сессии остаются JDBC
bank-target	Настоящие сервисы полного стека и банковские системы	Отдельные платформенные, модельные и банковские gates; схема не означает промышленный допуск

Примеры конфигурации: contracts/examples/profiles/*.json. Это не исполняемый Compose. Схема выбора: contracts/adapter-bindings.schema.json; полный каталог: contracts/technology-adapters.json. Профиль local-integrations допускает проверенные варианты с одним сервисом; каждый выбранный провайдер и его зависимость должны совпадать. Сам факт запуска контейнера не означает интеграцию приложения.

Неизвестный провайдер, отсутствующая способность, неподдержанный SDK или отсутствующий реализованный адаптер останавливают запуск соответствующего профиля. Смена провайдера выполняется при управляемом перезапуске. Автоматического переключения

источника сессий, финансов, KB или безопасных файлов при отказе нет. Восстанавливаемый кеш может дать miss с чтением разрешённого первоисточника.

3. Что заменяет каждую технологию

Технология и граница	Локальная реализация MVP	Настоящее подключение и предел замены
Qdrant / VectorIndexPort	NumPy: точный cosine search, metadata, allowlist точных версий	QdrantVectorIndexAdapter; отсутствуют ANN-масштабирование и распределённый индекс
Redis / SessionStorePort	PostgreSQL/JDBC: TTL, версия, отзыв сессии	RedisSessionAdapter; миграция с инвалидированием старых сессий, без fallback
Redis / CachePort	Ограниченный локальный TTL-кеш производных данных	RedisCacheAdapter; локальный кеш не общий между репликами
Feast / FeatureRepositoryPort и HistoricalFeatureReader	Общий transform, неизменяемые снимки в ML-схеме PostgreSQL	FeastFeatureRepositoryAdapter; registry/materialization/online store не эмулируются
MLflow / ModelRegistryPort	Content-addressed manifests, реальные evaluation reports и approval	MlflowModelRegistryAdapter; UI и распределённый tracking отсутствуют
Airflow / BatchJobRunnerPort	Явные offline-команды и журнал заданий с lease/idempotency	AirflowBatchJobRunnerAdapter; нет календарного scheduler/backfill
Evidently / QualityReportPort	Вычисленный DQ/метрики baseline, размер выборки и provenance	EvidentlyQualityReportAdapter; нет доказанного production drift на синтетике
vLLM / TextGenerationPort	Настоящий llama-server + Qwen GGUF на CPU	VllmGenerationAdapter; отдельная проверка модели/формата/GPU, без обещания битового паритета
Kafka / EventTransportPort	PostgreSQL outbox/dispatcher/inbox, доставка с повторами	KafkaEventTransportAdapter; outbox/inbox сохраняются, Kafka-протокол не эмулируется
S3 / ObjectStorePort	Версионные файлы quarantine/clean и metadata/hash в БД	S3ObjectStoreAdapter; локальный volume не обеспечивает multi-node durability
Prometheus / instrumentation	Реальный OpenMetrics endpoint и структурированные logs	Scrape/retention binding; exporter не заменяет хранилище и alerting Prometheus
Grafana / представление измерений	Локальный read-only отчёт с датой и evidence	Dashboard binding с настоящим datasource; отчёт не эмулирует Grafana
Kubernetes / развёртывание	Docker Compose для одного ПК	Манифесты, PV/secrets/probes/network policies; Compose не обеспечивает HA/multi-AZ

Порты вводятся для поведения приложения. Для Grafana/Kubernetes достаточно границы эксплуатации и конфигурации поставки: искусственные бизнес-интерфейсы ради названия технологии не нужны. Ограничения локального адаптера являются частью контракта, а не скрытой технической задолженностью.

4. Общие правила интерфейсов

Каталог содержит signatures, DTO, ownerRole/dataOwner, ограничения, правила переключения, contractCases и задачи. Существующие public/internal OpenAPI и строгие события остаются wire-источниками истины версии 1.4.0-design; документационная версия 1.5 не меняет API. Изменение DTO требует контрактного PR с примерами, а не локального исправления SDK-адаптера.

Для внешнего вызова задаются deadline, traceId и техническая identity; tenantScope вычисляет сервер. Классы ошибок порта: InvalidInput, NotFound, Forbidden, VersionConflict, Unavailable, DeadlineExceeded, ProviderNotImplemented. Адаптер переводит SDK-ошибки в этот набор; наружу не выходят connection string, query text и секреты. Бизнес API сохраняет ранее согласованные HTTP-ошибки; временный отказ ML ведёт к согласованному fallback.

Каждый порт получает одну общую группу проверок инвариантов и отдельные проверки конкретного сервиса. Общая группа проверяет допустимый результат, безопасность, версии и отказ, а не полную совместимость чужого протокола. Для настоящего провайдера дополнительно проверяются timeout/retry/connection pool/ресурсы и выбранные SDK/server версии. Неподключённый сервис имеет NOT_EXECUTED, а не PASS по локальной реализации.

5. Qdrant: реальный поиск в локальной обёртке

VectorIndexPort.search принимает нормализованный finite float32[384], indexDigest, список разрешённых chunkRef и topK от 1 до 10. ChunkRef связывает documentId, documentVersion и chunkId. Backend выдаёт ограниченный allowlist опубликованных источников и аудитории; пустой allowlist возвращает пустой результат. Фильтр применяется до ранжирования, затем проверяются citations и актуальный kbEpoch перед публикацией.

NumPy-адаптер действительно вычисляет cosine similarity на небольшом immutable корпусе. Qdrant-адаптер переносит те же ограничения в payload/ID-фильтр коллекции, но индекс не становится владельцем полномочий. Для SDK/API фильтров закрепить версию при P02/P27. Нельзя сначала искать по всем клиентам, а затем надеяться скрыть чужой ответ после генерации.

Переключение: подготовить коллекцию с теми же embedding/model/corpus digests, проверить количество и версии, фильтрацию и recall на фиксированном наборе, затем выбрать provider. ANN не обязан вернуть бит-в-бит тот же порядок; threshold recall@k объявляется до проверки и измеряется. На малом проверочном наборе можно использовать exact-режим; это не доказательство промышленной скорости. Отозванные знания запрещаются authority независимо от версии индекса.

6. Redis: раздельно сессии и кеш

SessionStorePort.resolve проверяет sessionHash, expiry и authEpoch; revoke использует ожидаемую версию. Локальные сессии сохраняются в JDBC, отзыв переживает restart. Redis-провайдер обязан сохранять смысл TTL/отзыва и отказывать при недоступном authority. При потере Redis нельзя автоматически читать старые JDBC-сессии: так вернулась бы уже отозванная сессия. Плановый переход инвалидирует активные сессии и требует повторного входа.

CachePort хранит только восстанавливаемые производные значения. Ключ учитывает namespace, sourceVersion, authEpoch и companyId, если данные компании участвуют в значении. TTL и максимальный размер обязательны. Cache miss/отказ Redis разрешает чтение того же разрешённого снимка из первоисточника; права, решения и idempotency records не переносятся в кеш. В дополнительном профиле сначала подключаем именно Redis-кеш, оставляя сессии JDBC.

7. Feast: признаки и исторические данные

FeatureRepositoryPort.create получает неизменяемый финансовый снимок и transformDigest, возвращает существующий FeatureSnapshot. get читает точный snapshotId/digest. Источник финансов и кредитное решение остаются на стороне банка; ML владеет схемой и преобразованием признаков. Повтор одного submissionSnapshotId/codeDigest не создаёт новый смысловой снимок.

HistoricalFeatureReader принимает entities с asOf и схему признаков; каждый источник должен быть доступен на момент среза: availableAt <= asOf. Локальный адаптер читает persisted provenance, использует общий batch/online transform и проверяется на утечки будущего. Для UCI нет надёжных календарных availableAt: это ограничение baseline, а не повод выдумать временную валидацию.

Feast различает historical retrieval и online retrieval. Online latest не заменяет point-in-time join и не меняет признаки уже отправленной заявки. Target-адаптер закрепляет feature definitions/registry, источник, правила materialization и отдельный online store. Даже с Feast результат, использованный в скоринге, сохраняется как immutable snapshot с digest.

8. MLflow, Airflow, Evidently и vLLM

MLflow-порт локально читает approved content-addressed bundle и записывает реальные evaluation reports. Registry alias разрешается в неизменяемую версию до активации; controller отдельно проверяет approval и включает релиз. Переименование alias не должно менять уже работающую модель. Инференс не зависит от доступности UI registry.

Airflow-порт принимает только allowlisted offline jobType и inputDigest/requestKey. Журнал хранит QUEUED/RUNNING/SUCCEEDED/FAILED, attempt, lease и hashes результатов. Подготовка признаков, обучение baseline, построение индекса и оценка запускаются явно вне клиентского запроса. Успех задания не даёт права автоматически опубликовать модель.

Evidently-порт локально считает missingness, объём данных и доступные метрики baseline. При отсутствии зрелых labels или достаточной выборки возвращает INSUFFICIENT_DATA. На синтетическом показе не объявляем доказанное качество или drift. Для настоящего Evidently закрепляются версии отчёта, окна и thresholds; общие численные показатели сравниваются отдельно от UI.

vLLM-порт сохраняет интерфейс генерации с approved context, bundleDigest, promptDigest, deadline и maxTokens. CPU llama.cpp выполняет реальный inference. vLLM требует отдельного проверенного формата артефактов, tokenizer/revision/precision и подходящего GPU; GGUF bundle не обещается загрузить тем же бинарным файлом. При смене engine повторяются golden/injection/citation проверки. Timeout означает SEARCH_ONLY, а подготовленный текст допустим только в обозначенном test double.

9. Переключение и параллельная разработка

P01 фиксирует порты, DTO и synthetic fixtures. Backend реализует локальные session/cache/event/object адаптеры в P25; ML реализует feature/vector/generation в P11/P14; интегратор собирает manifest/job/quality/metrics bindings в P26 с review ML. Frontend работает через прежний клиентский API и MSW: названия инфраструктуры не появляются в клиентском пути.

Минимальный каркас настоящего адаптера: зависимость SDK выключена в local-mvp, интерфейс реализован явно отключённым классом, README с mapping и ограничениями, общая группа тестов помечена NOT_EXECUTED до подключения сервиса. Проверка выбора такого каркаса должна завершаться ProviderNotImplemented. Подключение работающего SDK и сервиса - отдельный проверяемый PR, а не переименование локального класса.

Порядок для любого реального сервиса: закрепить версии/лицензии → реализовать adapter → поднять сервис → прогнать общие и специальные проверки → перенести/проверить данные → выбрать binding → провести smoke/restart/failure → записать доказательство в release. Рискованный dual-write в authority запрещён; shadow-read допустим на синтетике без второго бизнес-эффекта. Rollback возвращает согласованный полный bundle и проверенный provider; отзыв прав/KB не откатывается.

Оценка обязательного объёма увеличена на 9 человеко-часов локальных адаптеров и их проверки. P27 добавляет ещё 8 часов для настоящих Redis-cache/Qdrant/MLflow и не входит в обязательную сумму; это оптимистичная оценка при знакомых SDK. Feast/Airflow и другие сервисы сохраняются в архитектуре и каркасах, их полноценное внедрение не скрывается под этими 8 часами.

10. Как представить жюри

На схеме показать три колонки: именованный компонент стека → порт/граница → выбранный локальный или настоящий адаптер. Затем продемонстрировать один сквозной вызов, например RAG: KB allowlist → VectorIndexPort → NumPy cosine → Qwen → проверка источников. Для Redis показать сессию после restart и cache miss; для Feast - immutable feature snapshot и provenance. Так видны и архитектура, и фактическое поведение.

Для каждого компонента release-отчёт содержит capability, intendedTechnology, selectedProvider, implementationStatus, testStatus, commit, artifact/config digest и evidence path/hash. Метки: ARCHITECTURAL_SLOT, LOCAL_SUBSTITUTE, REAL_CONNECTED, TEST_DOUBLE. REAL_CONNECTED допускается только после реального интеграционного прогона и доказательства; отсутствие результата явно обозначается NOT_EXECUTED. Сам provider manifest не доказывает работоспособность.

Рекомендуемая формулировка: «Архитектура предусматривает Qdrant, Redis и Feast через стабильные границы. На этом стенде векторный поиск выполняет NumPy-адаптер, сессии сохраняются в PostgreSQL, признаки - в версионированных снимках. Вот действующий путь и проверки; вот конкретный mapping подключения сервисов». Успешную демонстрацию установленного Qdrant можно назвать реальной интеграцией только если приложение действительно выбрало его адаптер.

Источники и комплект

Различия historical/online: Feast feature retrieval. Фильтрация индекса: Qdrant filtering. Java SDK: Redis Lettuce. Неизменяемая привязка registry: MLflow workflows. Проверено 20.10.2026; проектный mapping и выбор адаптеров являются нашими архитектурными решениями. Точная совместимость SDK/server версий закрывается при реализации.

Машинный каталог, схема конфигурации, примеры профилей, план, Git и защита, редактируемый атлас.